Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 233 (2024) 623–637

Procedia Computer Science

www.elsevier.com/locate/procedia

5th International Conference on Innovative Data Communication Technologies and Application (ICIDCA 2024)

Flask-Based ASR for Automated Disorder Speech Recognition

Devalla Bhaskar Ganesh[a],Yellamma Pachipala[a],Syed Sania Rizvi[c],Teena Chowdary Manne[d],Himavanth Swamy Atchi[e],V V R Maheswara Rao[f]

corresponding author: Yellamma Pachipala

[a,b,c,d,e]*Dept of CSE, Koneru Lakshmaiah Education Foundation,Vaddeswaram, India*

[f]*Dept of CSE,Shri Vishnu Engineering College for Women(A),Bhimavaram,India*

Abstract

Speech disorders encompass a diverse range of conditions hindering effective communication, stemming from developmental, neurological, or physical factors. These challenges impact daily life and opportunities. Automatic Speech Recognition (ASR) technology emerges as a pivotal solution. ASR, powered by sophisticated algorithms, transcribes spoken language into written text, transcending traditional communication barriers. Tailorable to accommodate unique speech disorders, ASR offers the promise of empowering individuals to articulate their thoughts. Our Python Flask-based ASR(FBASR) applications meticulously crafted to cater to individuals, including children, grappling with speech disorders. It provides a user-friendly platform. Users simply submit a speech audio file in WAV format, and the application transcribes it. Alongside recognized text, it furnishes vital audio metrics. Additionally, it generates visual audio signal and spectrogram representations, all stored efficiently for future reference. The application's profound advantage lies in granting a voice to individuals with speech disorders, facilitating education, employment, and healthcare interactions. Furthermore, it serves as a resource for research and therapy development, promising inclusivity and enriching lives. The proposed work concludes by discussing the future outlook of cloud-based ASR and identifying some of the key areas of research that need to be addressed to make cloud-based DSR systems more accurate, reliable, and accessible

© 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application

Keywords: Cloud computing, Speech recognition, Disorder speech recognition, Machine learning, Artificial intelligence, Natural language processing

1877-0509 © 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application
10.1016/j.procs.2024.03.252

1. Introduction

Cloud-based ASR is the task of automatically transcribing the speech of people with speech disorders using cloud computing platforms. Speech disorders are a diverse group of conditions that can affect communication in a variety of ways, including slurring, stuttering, and difficulty articulating certain sounds. Automatic Speech Recognition (ASR) in the cloud has the potential to transform the way people communicate with speech disorders by providing real-time, precise transcription [1]. There are several benefits to cloud-based as-a-Service (ASR) systems over on-premises ones. The first is that they are more scalable, meaning they can be accessed anywhere in the world with a connection to the internet. The second is that they are able to be updated more often with new information and models. The third advantage is that they are also more cost-efficient, since users pay only for the resources they use. While there are many benefits to cloud ASR, it is important to note that there are also challenges[2]. One of the greatest challenges is the sheer volume of data and models needed. Speech disorders come in many forms and varieties, and there's no one-size-fits-all model that can rewrite the speech of every individual with a speech disorder [3].

To build high-quality ASR systems researchers need massive speech datasets from individuals with a broad spectrum of speech disorders [4]. Privacy and security are another issue that arise with cloud-based as-a-service (ASR) systems. When people use a cloud ASR system, they're uploading their speech data into the cloud. It's important to make sure that this data is secure from third-party access [5]. Cloud ASR systems should implement strong encryption and other safeguards to protect user data.Despite the difficulties, cloud based automated speech recognition (ASR) is developing quickly and has the potential to significantly improve the lives of individuals with speech impairments[6].

2. Literature Review

Automatic speech recognition in the cloud requires large datasets and models that can accurately transcribe speech of people with a variety of speech disorders [7]. Despite these challenges, cloud based DSR systems promise to transform the way people communicate with speech disorders by providing real-time, accurate speech transcription.

Challenges of cloud-based ASR

One of the major difficulties of cloud-based automatic speech recognition (ASR) is the large datasets and models required. Speech disorders are extremely heterogeneous and there is no one-size-fits-all model that accurately transcribes the speech of all individuals with speech disorders [8]. In order to create an accurate ASR system, researchers will need to collect large-scale datasets of speech from individuals with a wide variety of speech disorders. Collecting these datasets can be challenging and costly. It is also important to ensure that these datasets represent the entire population of people with a speech disorder. Privacy and security are another major challenge [9]. When individuals use cloud-based automated speech reporting (ASR) systems, they are transferring their speech data into the cloud. Speech data is sensitive and private information, and it must be protected from unauthorized access [10]. Cloud-based ASR systems should use strong encryption and other security measures to protect user data.

Applications of cloud-based ASR

Cloud based Automatic Speech Recognition (ASR) systems can be used for a wide range of purposes, including:

Education: Automatic Speech Recognition (ASR) systems can be used in the cloud to provide students with speech disorders with real-time transcription in the classroom, allowing them to be more involved in class discussions and work assignments.

Employment: With cloud-based automated speech recognition (ASR) systems, you can offer real-time speech recognition to employees with speech impairments in the office, helping them communicate better with colleagues and clients.

Healthcare: Additionally, cloud-based ASR systems can be used to allow speech-impaired patients to be recorded in real-time in doctors' offices and other healthcare settings. This can enhance patient-to-patient communication and enhance quality of care.

Speech Recognition-:

Speech recognition, often known as automatic speech recognition (ASR), stands as a fundamental technology within the domain of natural language processing. Its importance extends to a wide array of applications, encompassing services that transform spoken dialogues into written text, as well as the ubiquitous voice-activated virtual assistants such as Siri and Alexa [11]. At its core, speech recognition serves as the bridge between human communication and machine comprehension. This technology hinges on complex algorithms and models that dissect the intricate acoustic patterns of spoken language, capturing the subtleties of phonetic nuances and linguistic structures. Typically, a speech recognition system comprises two key components: an acoustic model that learns the acoustic features linked to phonemes or sub-word units, and a language model that forecasts word sequences based on linguistic probabilities. Through the accurate transformation of spoken words into written text, speech recognition systems facilitate seamless human-computer interactions, enhancing accessibility and efficiency in information retrieval and task execution across various industries and daily activities.

Over the past years, the progression of speech recognition technology owes much to the development of deep learning techniques, with a notable focus on neural networks like Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and the transformative Transformers architecture. These deep learning models have brought substantial enhancements to the precision and resilience of ASR systems. Acoustic models founded on neural networks are adept at discerning intricate speech patterns, while neural language models excel in grasping contextual cues and language semantics.

Moreover, the influence of speech recognition goes beyond mere convenience; it holds the potential to break down language barriers and advance inclusivity on a worldwide level.

Table 1: Comparison of different speech recognition approaches

Approach	Strengths	Weaknesses
Hidden Markov Models (HMMs) [9]	Simple and efficient, can be trained on large amounts of data	Less accurate than deep learning-based approaches, not good at handling noise
Deep neural networks (DNNs) [3]	More accurate than HMMs, can handle noise and variability in speech	More computationally expensive to train and deploy
End-to-end (E2E) models [10]	Most accurate speech recognition models, can be trained directly on audio data without the need for handcrafted features	More computationally expensive to train and deploy than DNNs

As illustrated in Table 1, the TIMIT Acoustic-Phonetic Continuous Speech Corpus serves as a standard dataset for assessing automatic speech recognition systems. This dataset encompasses recordings from 630 speakers representing 8 dialects of American English, with each speaker reading 10 phonetically-rich sentences.

LibriSpeech: The LibriSpeech dataset is a substantial collection of English speech recordings, comprising more than 1,000 hours of spoken content from over 1,000 different speakers who have read audio books. This dataset is frequently employed for the training of deep learning-based speech recognition systems.

Switchboard: The Switchboard corpus encompasses telephone conversations among native English speakers and stands as one of the most extensive sources of spontaneous speech. This corpus is commonly harnessed for training speech recognition systems in various applications, including voice transcription and automation in call centres.

Common Voice: The Common Voice dataset comprises speech recordings contributed by individuals from various regions globally, reading sentences in their respective native languages. This publicly accessible dataset is widely employed for training speech recognition systems that cater to diverse languages.

Table 2: Examples of speech recognition datasets

Dataset	Size	Description
TIMIT	6.3 hours	Recordings of 630 speakers of 8 dialects of American English reading 10 phonetically-rich sentences
LibriSpeech	960 hours	Recordings of 1,000 speakers reading audiobooks
Switchboard	2,400 hours	Recordings of telephone conversations between native English speakers
Common Voice	10,000 hours	Recordings of people from all over the world reading sentences in their native language

In Table 2 represents, Voice assistants like Siri, Alexa, and Google Assistant represent some of the most widely embraced applications of speech recognition technology. These voice-activated systems empower users to manage their devices and retrieve information through spoken directives. For instance, users can employ voice assistants to initiate tasks such as playing music, setting alarms, obtaining directions, and placing phone calls.

Voice transcription: Voice transcription involves the conversion of spoken audio into written text. This application of speech recognition technology is useful for transcribing various forms of spoken content, including dictation, lectures, meetings, and more. Voice transcription finds utility in generating written transcripts of these recordings, serving purposes like documentation, archiving, and enhancing accessibility.

Voice search: Voice search entails the capacity to seek information on the internet through vocal commands. This process leverages speech recognition technology to transform the user's spoken inquiry into written text, subsequently initiating a search request. Voice search offers a convenient method for accessing information while on the move or for users who encounter challenges with traditional typing.

Voice control: Voice control encompasses the capability to manage devices through vocal commands. This process relies on speech recognition technology to translate the user's spoken directives into actions that the device can execute. For instance, users can employ voice control to power on their TV, modify the volume, and choose channels. Voice control offers a convenient means of device management, eliminating the need for physical buttons or remote controls.

Apart from these all-encompassing speech recognition applications, there are also numerous specialized applications tailored to specific domains where speech recognition technology is employed. For instance, this technology finds utility in the following contexts:

Medical speech recognition: Speech recognition technology is harnessed for tasks such as transcribing medical records, verbalizing medical reports, and managing medical equipment.

Legal speech recognition: Speech recognition technology is applied for tasks such as converting legal proceedings into text, verbally generating legal documents, and conducting searches within legal databases.

Financial speech recognition: Speech recognition technology is utilized in tasks involving the transcription of financial transactions, verbal creation of financial reports, and management of financial trading systems.

Education: Speech recognition technology serves in the development of interactive educational materials, the conversion of lectures and meetings into text, and offering feedback to students.

Customer service: Speech recognition technology is employed for the transcription of customer calls, the automation of customer service responsibilities, and delivering multilingual customer support.

Table 3: Examples of speech recognition applications

Application	Example
Voice assistants	Siri, Alexa, Google Assistant

Voice transcription	Transcribing dictation, lectures, or meetings
Voice search	Searching for information on the internet using voice commands [5]
Voice control	Controlling devices such as smart TVs and speakers using voice commands

Word Error Rate: Word error rate (WER) stands as an assessment metric for gauging the efficacy of a speech recognition system [12]. It is computed as the proportion of errors within the recognized transcript when contrasted with the reference transcript. These errors encompass word substitutions, insertions, or deletions.

$$WER = \frac{(s+d+i)}{n} \quad (1)$$

WER is a frequently employed metric to assess the effectiveness of speech recognition systems, with lower values signifying higher accuracy. For instance, a WER of 10% implies that the system accurately recognized 90% of the words within the reference transcript.

WER can be influenced by various factors, as outlined in [14]:

* The audio input's quality
* The intricacy of the recognized language
* The size and quality of the training data used to train the speech recognition system [13]
* The presence of background noise in the audio input

When selecting a speech recognition system for a specific application, it is crucial to take into account the WER as a significant metric. For instance, a speech recognition system intended for transcribing dictation should ideally possess a lower WER compared to a speech recognition system designed for managing a voice assistant [14].

Here are some examples of WER for popular speech recognition systems:

* Google Cloud Speech-to-Text: 5-10%
* Amazon Transcribe: 6-12%
* Microsoft Azure Speech Services: 7-13%

WER remains a dynamic field of study, with ongoing research efforts aimed at creating novel speech recognition systems capable of attaining even further reductions in WER.

Word Recognition Rate-:

Word recognition rate (WRR) serves as a metric for assessing the precision of a speech recognition system in accurately identifying individual words.

WER remains a dynamic field of study, with ongoing research efforts aimed at creating novel speech recognition systems capable of attaining even further reductions in WER.

Word Recognition Rate-:

Word recognition rate (WRR) serves as a metric for assessing the precision of a speech recognition system in accurately identifying individual words.

$$WRR = 1 - WER = \frac{(n-s-d-i)}{n} = \frac{h-i}{n} \quad (2)$$

$$h = n - (s+d) \quad (3)$$

WRR shares a close connection with word error rate (WER); however, their calculations differ. WER encompasses all types of errors, such as substitutions, insertions, and deletions of words, while WRR exclusively accounts for word substitutions.

WRR proves to be a valuable metric for appraising the effectiveness of speech recognition systems in contexts where precise recognition of individual words holds significance, as observed in applications like voice transcription

and voice search. Table 4 displays a comparison between WER and WRR, offering examples of WRR values for well-known speech recognition systems.
Google Cloud Speech-to-Text: 95-98%
Amazon Transcribe: 94-97%
Microsoft Azure Speech Services: 93-96%

Table 4: Comparison between WER and WRR

Metric	Definition
WER	Percentage of errors in a recognized transcript compared to the reference transcript
WRR	Percentage of words that are correctly recognized in a transcript, compared to the reference transcript

3. Proposed Work

Automatic Speech Recognition (ASR) stands as a transformative technology pivotal in the conversion of spoken language into written text . It functions by processing audio inputs, which can stem from diverse sources such as microphones, telephone conversations, or recorded files. The initial step involves audio preprocessing to improve its quality, encompassing tasks like noise reduction and filtering. Subsequently, ASR systems extract pertinent features from the audio, typically employing methods like Mel-frequency cepstral coefficients (MFCCs) and filter banks. These extracted features form the foundation for acoustic models, which undergo training using extensive datasets comprising transcribed audio and phonetic data.

ASR technology is applied across various domains, including transcription services, voice-activated virtual assistants, customer service automation, accessibility enhancements for individuals with hearing impairments, and voice-controlled search functions.It has transformed the way we engage with technology, becoming an integral component of our digital experiences

Speech Production-:

An excitation, originating in the lungs, materializes as an initial waveform, representing the flow of air over a specific duration.Subsequently vocal cord vibrations are generated, and various filters, such as those formed by the pharynx and tongue, are then applied to this waveform.

The resulting signal generated can be expressed as:

$$s = f * e \tag{4}$$

outcome of convolving the excitation with the filters, under the assumption that f is linear and time-independent.

$$s(t) = \int_{-\infty}^{+\infty} e(t) f(t - T) dT \tag{5}$$

Starting from the initial waveform, we directly capture the glottal spectrum, which emerges from the vocal cords. As we move further up the vocal tract, particularly at the pharyngeal level, pitch variations occur, giving rise to the

formation of formants in the vocal tract. In the end, this resulting spectrum provides a representation of the intensity across the frequency spectrum generated.

The normally, gauge the efficiency of an Automatic Speech Recognition (ASR) system by evaluating its performance through the Word Error Rate (WER). These discrepancies can manifest in different types of inaccuracies, including:

- Substitutions, N_{SuB}, occur when a word is replaced
- Insertions, N_{INS} , involve adding a word that was not originally pronounced
- Deletions, N_{DEL} , refer to instances where a word is omitted from the transcript.

The Word Error Rate (WER) is calculated as follows:

$$WER = \frac{N_{SuB}+N_{INS}+N_{DEL}}{|N_{words-transcript}|} \quad (6)$$

In an ideal situation, the Word Error Rate (WER) should strive for a value approaching 0, signifying a flawless match. The calculation of substitutions, insertions, and deletions is carried out using the Wagner-Fischer dynamic programming algorithm for word alignment.

In the statistical historical approach to ASR:

We can denote the most fitting word sequence as W^*, chosen from the available vocabulary. At the same time, we assign a label to the input sequence of acoustic features as X. Statistically, our goal is to determine the sequence that optimally meets the following criteria:

$$W^* = argmax_W P(W|X) \quad (7)$$

This is often referred to as the "Foundational Equation of Statistical Speech Processing." Utilizing Bayes' Rule, we can express it as follows:

$$W^* = argmax_w \frac{P(X|W)P(W)}{P(X)} \quad (8)$$

Lastly, assuming independence and omitting the term P(X), we can restate our problem as:

$$W^* = argmax_w P(X|W)\text{P(W)} \quad (9)$$

Where:

- $argmax_w$ is the search space, a function of the vocabulary
- $P(X|W)$ is called the acoustic model
- $P(W)$ is called the language model

Acoustic Model: HMM-GMM Acoustic Model

The acoustic model, often employing sophisticated techniques like Hidden Markov Models (HMMs) and Artificial Neural Networks, is crafted to capture the intricate connection between the audio signal and the phonetic components in the language.

In the realm of isolated word or pattern recognition, the acoustic features (referred to here as Y) function as inputs to a classifier, tasked with producing the accurate word as the output

The acoustic model goes beyond fundamental classification, as it produces a sequence of phonemes.

The shown the Hidden Markov Models are particularly well-suited for Acoustic Models due to their proficiency in modeling sequential data. Within the HMM framework, there exist underlying states denoted as s_i , and at each of these states, corresponding observations O_i are generated.

The realm of Hidden Markov Models (HMMs), a solitary phoneme is commonly represented by a linear HMM, comprising 3 or 5 states, typically aligning with the phoneme's commencement, intermediate, and conclusion stages.

The Hidden Markov Model (HMM) postulates a kind of independence among observations, where:

$$P(O_t=x|s_t=q_i, S_{t-1}=q_j, \ldots)=P(O_t=x|s_t=q_i) \quad (10)$$

The Hidden Markov Model (HMM) has the capacity to generate context-dependent phonemes, known as triphones, which consist of a group of three phonemes. The left and right phonemes within a triphone represent the left and right context, respectively. The training of HMMs is accomplished using the Baum-Welch algorithm. During this process, HMMs acquire the ability to estimate the probabilities associated with the initiation and culmination of phonemes at a specific time, denoted as "t." It is widely assumed that observations are generated by a blend of Gaussians, often implemented as Gaussian M"xt"re Models (GMMs) at each state, indicating:

$$P(O_t=y|s_t=q_i)=P(X|W)=\sum_{m=1}N(y,\mu jm,\sum jm) \quad (11)$$

The training process of the HMM-GMM system is tackled using the Expectation Maximization (EM) method. Within EM training, the outputs of the GMM, referred to as P(X|W), are utilized as inputs for iterative GMM training.

ALGORITHM: PROPOSED FLASK-BASED AUTOMATIC SPEECH RECOGNITION (FBASR)

Importing Libraries

This section initiates by importing essential Python libraries, a pivotal step in any software development endeavor. Each library serves a distinct role in enhancing the script's capabilities. To commence, it imports the `speech_recognition` library under the alias `sr`, a core component for speech recognition from audio files. This library abstracts the intricate task of transmuting audio data into text, rendering it accessible and user-friendly for developers. Following that, the script introduces `numpy` as `np`, a key asset for various numerical operations encompassing audio data management and mathematical computations. This library equips developers with potent tools for efficient data manipulation and analysis. The inclusion of `matplotlib. pyplot` facilitates the creation of visual representations for audio data, including audio signals and spectrogram graphs, serving as invaluable aids in visualizing and further dissecting the audio content. Moreover, `scipy.io. wavfile` is employed for seamless audio file handling, enabling the extraction of crucial audio attributes like sample rate and bit depth. In addition, the imports of `io` and `base64` are instrumental for working with memory buffers and encoding images, particularly for saving the generated graphs as base64-encoded strings. Lastly, the `mysql. connector` library is enlisted, empowering the script to establish connections and interact with a MySQL database. This element plays a central role in storing the recognized text and pertinent audio details.

Speech Recognizer

The initiation of the speech recognizer holds pivotal significance within the script's functionality, as it lays the fundamental groundwork for the core speech recognition process. In this section, a speech recognition engine is created by instantiating the `Recognizer` class derived from the `speech_recognition` library, and it is assigned to the variable `recognizer`. This recognition engine forms the central component responsible for processing audio data and its transformation into text. It offers a standardized interface encompassing various speech recognition services, including Google's speech recognition API. The creation of a `Recognizer` instance is paramount for multiple reasons. It orchestrates the acquisition and manipulation of audio data derived from the provided audio file. It adeptly manages various audio file formats and conversions, along with data streaming and segmentation, thereby enabling an efficient and effective speech recognition process.

Recognize Speech

The "Recognize Speech" segment of the script is the pivotal juncture where the core speech recognition procedure unfolds. It commences by accessing the designated audio file specified as the input parameter, accomplished using the `AudioFile` class sourced from the `speech_recognition` library. This initial step prepares the audio data for the recognition process. Enclosed within a try-except block, the script undertakes the task of speech recognition, utilizing Google's speech recognition service. This process entails the transmission of the audio data to the service, which in turn processes it and furnishes the recognized text as output.

text
Australia team Char matchon ki test series ke liye India mein hai
Australia team Char matchon ki test series ke liye India mein hai
Australia team Char matchon ki test series ke liye India mein hai
p a p e o o
Australia team Char matchon ki test series ke liye India mein hai
Australia team Char matchon ki test series ke liye India mein hai
Australia team Char matchon ki test series ke liye India mein hai
p a p e o o
p a p e o o
p a p e o o
iski khasiyat hai ki yah ek degree Celsius mein bhi kharab Nahin Hoti
iski khasiyat hai ki yah ek degree Celsius mein bhi kharab Nahin Hoti

Fig .1. Recognized Text from Disorder Speech

In figure 1 shown, the proposed work dig deeper and extract key features from the audio data. It involves extracting important features like sample rate, number of channels, decibel level, bit depth, and audio length. These features provide valuable insights into the intrinsic properties of the audio data and are important for various applications. For example, the sample rate helps us understand the duration and quality of the audio. The decibel level tells us about the intensity of the audio. And the bit depth gives us insights into the accuracy of the audio.

Analyze Audio Characteristics

The "Analyze Audio Characteristics" segment within the script assumes a critical role by delving into the technical intricacies of the audio data. It initiates the process by extracting several audio attributes, each of which imparts valuable insights into the essence and quality of the audio content. To begin with, it determines the sample rate, signifying the number of audio samples captured per second. This parameter holds substantial significance in defining the audio's quality and can significantly impact both its playback and analytical aspects. A higher sample rate typically leads to enhanced audio fidelity, albeit with larger file sizes. The script's retrieval of this detail empowers users to gauge the level of intricacy within the audio data. Subsequently, it ascertains the number of audio channels, indicating whether the audio is mono or stereo. This information proves pivotal, as it can influence the perceptual experience of the audio and the manner in which it is processed. For instance, stereo audio, featuring two channels, facilitates a more immersive listening encounter. By extracting this characteristic, the script provides insights into the audio's spatial attributes. The script then proceeds to calculate the decibel level, serving as a metric for audio intensity. It employs the maximum absolute amplitude of the audio to deduce this measure

Generate Audio Signal Graph

The “Generate audio signal graph” segment in the script plays an important role in converting the audio data to a waveform, which is a graphical representation of the amplitude over time. This graph allows you to interpret the temporal attributes of the audio data. To create an audio signal graph in the script, you can use the tools of Matplotlib. Matplotlib’s ‘plot’ function allows you to construct the audio signal in a continuous line chart. The X-axis represents the time in seconds, and Y-axis represents the amplitude of your audio samples. The resulting graph gives you a visual representation of the audio’s intensity over time. It helps you to analyze the audio data to find patterns, variations and trends. This graph can reveal important information, such as whether there is speech, music or silence. In addition, it can also explain the dynamic parts of the audio, such as the loud and the quiet parts. This visual representation is very helpful in understanding the content and the structure of the audio

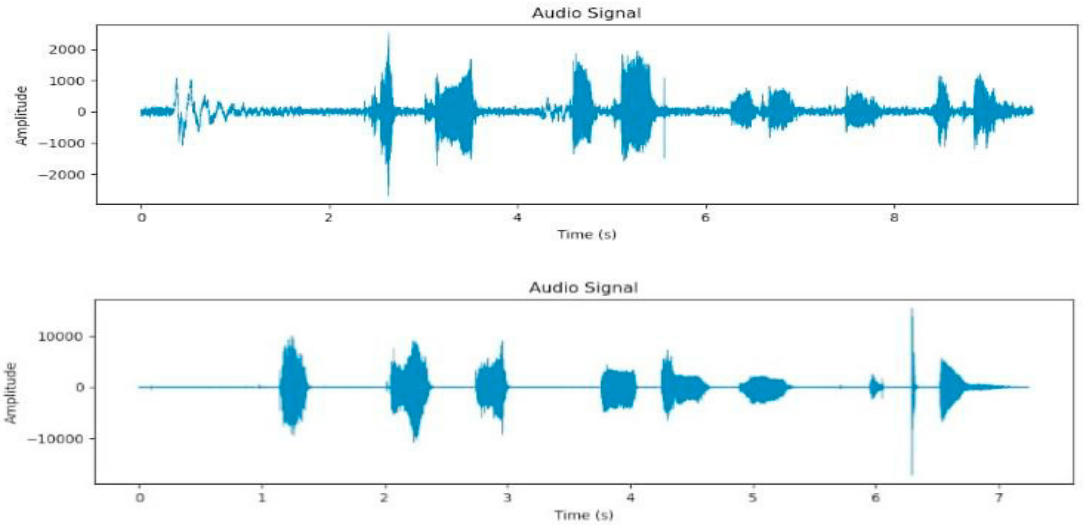


Fig.2.Audio Signal of a WAV File

Recognized Text: p a p e o o
Sample Rate: 22050 Hz
Number of Channels: 1
Decibel: 68.60795021057129 dB
Bit Depth: 16 bits
Audio Length (seconds): 9.47374149659864

Recognized Text: h e t e
Sample Rate: 22050 Hz
Number of Channels: 1
Decibel: 84.7594165802002 dB
Bit Depth: 16 bits

This script uses Matplotlib's functions to define the graph's title as "Audio Signal", label the X axis as "Time (s)" and the Y axis as "Amplitude".

Convert Graph to Base64

The most important part of the script is the "Convert graph to Base64" segment. The converted graph image is converted into a Base64 encoded string. Base64 is a string format that converts binary data to text. The converted audio signal graph is suitable for storing in a database or for integration into a web page. To effect this conversion, the script saves the graph generated by Matplotlib as an image within an in-memory buffer facilitated by the `io. BytesIO` class. The graph is usually saved in a file format, usually PNG, which is known for being lossless. The use of in-memory buffers means that the image does not need to be written to the filesystem, which improves performance and security. Upon preserving the graph as an image within the buffer, the script proceeds to convert it into a base64-encoded string, accomplished through the `base64.b64encode` function. This encoded string becomes well-suited for storage within a database or embedding in documents and web pages, rendering it easily shareable and transferable.

Generate Spectrogram Graph

The Generate Spectrogram Graph section of the script is essential for understanding and visualizing the frequency content of the audio data over time. Spectrogram graphs are powerful tools for understanding and understanding the spectral properties of the audio, such as the presence of particular frequencies, frequency content variations, and frequency intensity at different time points. To create the spectrogram graph, the script employs Matplotlib's `plt. specgram` function. This function calculates and presents the spectrogram of the audio signal, which is essentially a time-frequency representation of the audio. It breaks the audio signal down into small time segments and calculates

the frequency content in each segment using a heatmap like representation. X-axis represents the time (seconds) and Y-axis the frequency (Hz). The color intensity represents the intensity of the frequency at a particular point in time. This visual representation allows users to recognize patterns such as frequency shifts over time, presence of particular harmonics or elements, and overall spectral composition of audio. Spectrograms are used in various fields such as speech recognition, music processing and sound recognition because they provide a comprehensive view of an audio's frequency dynamics. In the script, it improves the interpretability of a spectrogram graph by adding labels, titles and color bar.

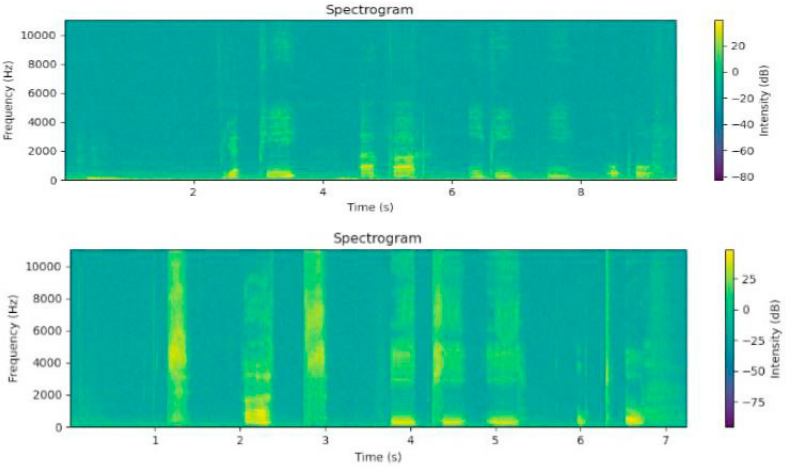


Fig.3.Spectogram of a WAV File

Recognized Text: p a p e o o
Sample Rate: 22050 Hz
Number of Channels: 1
Decibel: 68.60795021057129 dB
Bit Depth: 16 bits
Audio Length (seconds): 9.47374149659864

Recognized Text: h e t e
Sample Rate: 22050 Hz
Number of Channels: 1
Decibel: 84.7594165802002 dB
Bit Depth: 16 bits

In short, the addition of these elements improves the visual clarity of the graphical representation, making it easier for users to understand and interpret the frequency content of the audio.

4. Result and Analysis

The "Close the graphs" section in the script plays an important role in managing system resources and controlling the user interface. This section explicitly closes the graphs generated by Matplotlib, whether audio signal graphs or spectrogram graphs.

Closing the graphs ensures that all allocated memory is immediately freed and that the graphical representations are removed from the UI. This step is important to avoid resource leaks and to maintain the responsiveness and efficiency of the script, especially when dealing with many audio files or when dealing with a large application. This is particularly relevant when the script operates within a graphical environment or in automated scenarios where visual feedback is unnecessary. It prevents the accumulation of graphs in memory, which can lead to performance degradation or excessive memory consumption over time. The proper closure of graphs contributes to the overall stability and efficiency of the script, advocating responsible resource management. In cases where the script is part

of a larger application or runs as a background process, leaving graphs open indefinitely can result in the unwarranted consumption of system resources.

Save Results to MySQL Database

The "Save Results to MySQL Database" section is a critical component of the script, responsible for permanently storing the recognized text, audio characteristics, and base64-encoded graph images in a MySQL database. This process ensures that the valuable information obtained from the audio file and generated during the analysis is securely retained for future reference and retrieval. In this section, we will connect to the MySQL database using the following basic connection parameters: Host, user credentials, target database, cursor object, SQL query, data manipulation, SQL INSERT query, and more. The SQL INSERT query defines the structure of data to be entered into the “results” table. This includes columns for text data, sample rate data, number of channels data, decibel level data, bit depth data, audio length data, and base64 encoded representation of the audio signal data and spectrogram data. The data entered into the table is shown as a tuple. Once the database is connected, the script will execute the INSERT query, including the data provided, and make changes to the database accordingly. This step ensures that the audio analysis results are stored in a systematic and secure manner, making them available for future reference, report, or detailed analysis. This is an important step in maintaining a record of the audio processing and maintaining data integrity.

#	id	text	sample_rate	num_channels	decibel	bit_depth	audio_length	audio_signal_base64	spectrogram_base6
1	1	Australia team Char matchon ki tes...	16000	1	90.1371	16	3.8725	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
2	2	Australia team Char matchon ki tes...	16000	1	90.1371	16	3.8725	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
3	3	Australia team Char matchon ki tes...	16000	1	90.1371	16	3.8725	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
4	4	p a p e o o	22050	1	68.6079	16	9.47374	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
5	5	Australia team Char matchon ki tes...	16000	1	90.1371	16	3.8725	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
6	6	Australia team Char matchon ki tes...	16000	1	90.1371	16	3.8725	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
7	7	Australia team Char matchon ki tes...	16000	1	90.1371	16	3.8725	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
8	8	p a p e o o	22050	1	68.6079	16	9.47374	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
9	9	p a p e o o	22050	1	68.6079	16	9.47374	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
10	10	p a p e o o	22050	1	68.6079	16	9.47374	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
11	11	iski khasiyat hai ki yah ek degree C...	16000	1	90.1369	16	3.9775	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
12	12	iski khasiyat hai ki yah ek degree C...	16000	1	90.1369	16	3.9775	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
13	13	iski khasiyat hai ki yah ek degree C...	16000	1	90.1369	16	3.9775	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
14	14	p a p e o o	22050	1	68.6079	16	9.47374	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
15	15	ID photo	22050	1	83.6317	16	5.01551	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
16	16	as a easy	22050	1	83.5236	16	5.01551	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
17	17	Allah Allah	22050	1	87.818	16	5.57279	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
18	18	scissors fishing letters zebra	22050	1	88.7162	16	4.45823	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
19	19	scissors fishing letters zebra	22050	1	88.7162	16	4.45823	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
20	20	sheep snake from feather	22050	1	87.3501	16	4.45823	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
21	21	sheep snake from feather	22050	1	87.3501	16	4.45823	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
22	22	yellow strawberry spider	22050	1	88.1773	16	4.45823	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
23	23	p a p e o o	22050	1	68.6079	16	9.47374	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
24	24	iski khasiyat hai ki yah ek degree C...	16000	1	90.1369	16	3.9775	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
25	25	sheep snake from feather	22050	1	87.3501	16	4.45823	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...
26	26	yellow strawberry spider	22050	1	88.1773	16	4.45823	iVBORw0KGgoAAAANSUhEUgA...	iVBORw0KGgoAA...

Fig.4. Results saved in Database (MySQL)

Return Results

The "Return Results" section represents the final phase of the script and holds the responsibility of packaging and presenting the crucial outcomes of the audio analysis to the user or application. Following the processing of audio data, extraction of characteristics, graph generation, and storage of results in a database, this section constructs a well-organized data object that encapsulates the recognized text, audio properties, and base64-encoded graph images. To achieve this, the script assembles a Python dictionary containing various fields, including "text," "sample_rate," "num_channels," "decibel," "bit_depth," "audio_length," "audio_signal_base64," and "spectrogram_base64." These fields store the results of the analysis, ensuring convenient accessibility and utilization of the data. By structuring the information in a dictionary, the script maintains a well-defined and easily navigable format for the results, whether for returning to the caller or further utilization within the script. This structured return simplifies data management and interpretation, rendering it an indispensable component for users or applications that require interaction with the audio analysis results. The "Return Results" section not only offers the recognized text but also furnishes the technical attributes of the audio data and visual representations of the audio signal and spectrogram.

Handle Exceptions

The "Handle Exceptions" section of the script plays a vital role in ensuring the reliability and robustness of the audio analysis process. It is enclosed within a try-except block designed to address two specific types of exceptions that may occur during the script's execution: `sr.UnknownValueError` and `sr.RequestError`.Thefirstexception, `sr.UnknownValueError`, is raised when the speech recognition process fails to identify any speech content within the provided audio. In this case, the script returns the message "Speech not recognized." This handling approach effectively communicates to the user or calling application that the recognition attempt did not yield meaningful results. This empowers users to take appropriate action, such as reprocessing the audio file or gracefully managing situations where speech recognition is challenging or unsuccessful. The second exception, `sr. Request Error`, arises when issues occur with the recognition request itself.

Recognized Text

The recognized text, derived from the audio content provided, holds a central and immensely valuable role within the code's results. It serves as a representation of the spoken information contained in the audio and stands as the primary output of the script. The recognized text finds extensive utility across diverse domains, making it a pivotal element of the results.In transcription services, the recognized text plays a crucial role in the conversion of spoken language into a written format. This is essential for creating written records for a wide range of situations, such as meeting, interview, lecture, and any spoken communication. It is a time-saving and accurate alternative to manual transcription that ensures the efficient and reliable recording of spoken content.

The recognized text is also essential for voice-controlled applications such as voice assistants where it is used to understand and execute user commands. It enables natural language processing (NLP) applications to extract meanings from spoken language, allowing for functionalities such as voice search and Chatbots, as well as conversational interfaces.

The importance of recognized text extends beyond the realm of speech analysis and sentiment analysis. It plays an essential role in analyzing spoken data, helping organizations to gain insights into areas such as customer feedback and market trends, as well as customer satisfaction. It also helps in extracting meaningful patterns and sentiments. In essence, recognized text is a versatile output with multifaceted applications in transcription, voice-controlled systems, and data analysis, making it a valuable asset across a spectrum of use cases.

Audio Characteristics

The audio characteristics, forming an integral part of the code's output, offer vital technical insights into the processed audio data. These attributes are of utmost importance for comprehending the inherent qualities of the audio, and they hold significance across a spectrum of applications in the domain of audio processing and analysis. The sample rate, signifying the quantity of audio samples captured per second, stands as a fundamental audio characteristic. It serves as a linchpin for audio quality, exerting a considerable influence on playback and analysis. A higher sample rate typically results in superior audio fidelity, capturing finer audio details. Nevertheless, it also engenders larger file sizes. This characteristic plays a pivotal role in various audio applications, particularly in music production, where pristine audio quality is imperative. It also ensures compatibility between audio files and various playback systems. The number of audio channels, which discerns whether the audio is mono or stereo, ranks as another critical attribute. Mono audio comprises a single channel, while stereo audio encompasses two.

Spectrogram Graph

The spectrogram graph, an integral part of the code's output, serves as a visual depiction of the audio data's frequency content over time. This graphical representation offers pivotal insights into the audio's spectral attributes, rendering it a highly valuable component of the result. Spectrograms find extensive utility in diverse domains, including speech analysis, music processing, and sound recognition. They excel in unveiling specific frequencies within the audio, allowing the detection of patterns, fluctuations in frequency content, and the intensity of these frequencies across time. In the realm of speech analysis, spectrograms unveil distinctive features like formants and harmonics, substantially contributing to phonetic studies and speaker recognition. In music processing, they aid in the visualization of musical components, facilitating the identification of instruments and harmonic structures. Furthermore, spectrograms play a vital role in sound recognition applications, encompassing tasks such as the identification of particular sounds in audio streams, a function commonly employed in speech recognition systems and environmental sound analysis.

5. Conclusion

The Flask-Based ASR for Automated Disorder Speech Recognition research paper presents a pioneering approach to address the critical need for efficient disorder speech recognition. Through the integration of Flask, a micro web framework for Python, and Automatic Speech Recognition (ASR) technology, the study showcases a promising system capable of accurately identifying speech disorders in real-time. By leveraging Flask's flexibility and ease of deployment, the researchers have developed a robust platform that offers accessibility and scalability, essential factors for practical applications in healthcare and beyond.

In conclusion, the Flask-Based ASR for Automated Disorder Speech Recognition research paper demonstrates a compelling fusion of Flask's web framework capabilities with cutting-edge ASR technology, yielding a powerful tool for speech disorder identification. This innovative approach holds immense promise in revolutionizing the diagnosis and treatment of speech-related disorders, showcasing the potential for Flask-based solutions to drive significant advancements in healthcare and technology integration.

References

[1] C. -T. Do, D. Pastor and A. Goalic, "On the Recognition of Cochlear Implant-Like Spectrally Reduced Speech With MFCC and HMM-Based ASR," in IEEE Transactions on Audio, Speech, and Language Processing, vol. 18, no. 5, pp. 1065-1068, July 2010, doi: 10.1109/TASL.2009.2032945.

[2] L. K. Saul and M. G. Rahim, "Maximum likelihood and minimum classification error factor analysis for automatic speech recognition," in IEEE Transactions on Speech and Audio Processing, vol. 8, no. 2, pp. 115-125, March 2000, doi: 10.1109/89.824696.

[3] G. S. Meltzner, J. T. Heaton, Y. Deng, G. De Luca, S. H. Roy and J. C. Kline, "Silent Speech Recognition as an Alternative Communication Device for Persons With Laryngectomy," in IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 25, no. 12, pp. 2386-2398, Dec. 2017, doi: 10.1109/TASLP.2017.2740000.

[4] L. Chai, J. Du, Q. -F. Liu and C. -H. Lee, "A Cross-Entropy-Guided Measure (CEGM) for Assessing Speech Recognition Performance and Optimizing DNN-Based Speech Enhancement," in IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 29, pp. 106-117, 2021, doi: 10.1109/TASLP.2020.3036783.

[5] K. -S. Lee, "EMG-Based Speech Recognition Using Hidden Markov Models With Global Control Variables," in IEEE Transactions on Biomedical Engineering, vol. 55, no. 3, pp. 930-940, March 2008, doi: 10.1109/TBME.2008.915658.

[6] T. Claes, I. Dologlou, L. ten Bosch and D. van Compernolle, "A novel feature transformation for vocal tract length normalization in automatic speech recognition," in IEEE Transactions on Speech and Audio Processing, vol. 6, no. 6, pp. 549-557, Nov. 1998, doi: 10.1109/89.725321.

[7] Linkai Bu and T. . -D. Church, "Perceptual speech processing and phonetic feature mapping for robust vowel recognition," in IEEE Transactions on Speech and Audio Processing, vol. 8, no. 2, pp. 105-114, March 2000, doi: 10.1109/89.824695.

[8] M. S. Hawley ., "A Voice-Input Voice-Output Communication Aid for People With Severe Speech Impairment," in IEEE Transactions on Neural Systems and Rehabilitation Engineering, vol. 21, no. 1, pp. 23-31, Jan. 2013, doi: 10.1109/TNSRE.2012.2209678.

[9] C. Fan, J. Yi, J. Tao, Z. Tian, B. Liu and Z. Wen, "Gated Recurrent Fusion With Joint Training Framework for Robust End-to-End Speech Recognition," in IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 29, pp. 198-209, 2021, doi: 10.1109/TASLP.2020.3039600.

[10] Guangji Shi, M. M. Shanechi and P. Aarabi, "On the importance of phase in human speech recognition," in IEEE Transactions on Audio, Speech, and Language Processing, vol. 14, no. 5, pp. 1867-1874, Sept. 2006, doi: 10.1109/TSA.2005.858512.

[11] Z. -Q. Wang and D. Wang, "A Joint Training Framework for Robust Automatic Speech Recognition," in IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 24, no. 4, pp. 796-806, April 2016, doi: 10.1109/TASLP.2016.2528171.

[12] A. D. C. Chan, K. B. Englehart, B. Hudgins and D. F. Lovely, "Multiexpert automatic speech recognition using acoustic and myoelectric signals," in IEEE Transactions on Biomedical Engineering, vol. 53, no. 4, pp. 676-685, April 2006, doi: 10.1109/TBME.2006.870224.

[13] A. Huang, T. H. Falk, W. -Y. Chan, V. Parsa and P. Doyle, "Reference-free automatic quality assessment of tracheoesophageal speech," 2009 Annual International Conference of the IEEE Engineering in Medicine and Biology Society, Minneapolis, MN, USA, 2009, pp. 6210-6213, doi: 10.1109/IEMBS.2009.5334545.

[14] Karimunnisa, S., Pachipala, Y. Task Classification and Scheduling Using Enhanced Coot Optimization in Cloud Computing (2023) International Journal of Intelligent Engineering and Systems, 16 (5), pp. 501-511.

[15] Y. Takashima, T. Takiguchi and Y. Ariki, "End-to-end Dysarthric Speech Recognition Using Multiple Databases," ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Brighton, UK, 2019, pp. 6395-6399, doi: 10.1109/ICASSP.2019.8683803.

[16] Karimunnisa, S., Pachipala, Y. An AHP based Task Scheduling and Optimal Resource Allocation in Cloud Computing (2023) International Journal of Advanced Computer Science and Applications, 14 (3), pp. 149-159.

[17] Y. Matsuzaka, R. Takashima, C. Sasaki and T. Takiguchi, "Data Augmentation for Dysarthric Speech Recognition Based on Text-to-Speech Synthesis," 2022 IEEE 4th Global Conference on Life Sciences and Technologies (LifeTech), Osaka, Japan, 2022, pp. 399-400, doi: 10.1109/LifeTech53646.2022.9754798.

[18] G. S. Meltzner, J. T. Heaton, Y. Deng, G. De Luca, S. H. Roy and J. C. Kline, "Silent Speech Recognition as an Alternative Communication Device for Persons With Laryngectomy," in IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 25, no. 12, pp. 2386-2398, Dec. 2017, doi: 10.1109/TASLP.2017.2740000.

[19] T. A. M. Celin, T. Nagarajan and P. Vijayalakshmi, "Data Augmentation Using Virtual Microphone Array Synthesis and Multi-Resolution Feature Extraction for Isolated Word Dysarthric Speech Recognition," in IEEE Journal of Selected Topics in Signal Processing, vol. 14, no. 2, pp. 346-354, Feb. 2020, doi: 10.1109/JSTSP.2020.2972161.

[20] Ganesan, V., Sobhana, M., Anuradha, G., Yellamma, P., Devi, O.R., Prakash, K.B., Naren, J. Quantum inspired meta-heuristic approach for optimization of genetic algorithm (2021) Computers and Electrical Engineering, 94, art.